

Lord of the strings

"What if a math model had to explain its work like a medieval monk who's never heard of Hindu-Arabic numerals?"

What this task is

We ask you to fine-tune a small open-source language model with reinforcement learning so it learns to solve grade school math problems under a lipogram constraint.

You will not be graded on raw accuracy. The exercise is set up so the obvious recipe (copy a public GSM8K + GRPO Colab tutorial) does not quite work. We want to see how you diagnose, reshape, and reason through that.

A few definitions, in case any are unfamiliar:

- GSM8K — Grade School Math 8K. A standard dataset of ~8,500 math word problems with integer answers. HuggingFace id `openai/gsm8k`, config `main`. Each row has a question (English word problem) and an answer (a step-by-step solution ending in `#### <integer>`). The integer is the gold answer.
- RLVR — Reinforcement Learning from Verifiable Rewards. The reward is computed by a deterministic checker (regex + arithmetic), not by another LLM. This is the family of recipes used by DeepSeek-R1.
- Lipogram — A piece of writing that intentionally avoids one or more characters. Famous example: Georges Perec's novel *La Disparition*, 300 pages without the letter 'e'. Here the forbidden characters are the digits 0-9.

The constraint, precisely

Every output your trained model produces must satisfy all three of:

1. Correctness. The final integer answer must equal the GSM8K gold answer.
2. No-digits (the lipogram). Zero ASCII digits anywhere in the output. Every number — intermediate calculations, the final answer, anything — must be spelled in English. The forbidden character set is exactly `0 1 2 3 4 5 6 7 8 9`.
3. Format. The output must end with one line of the form

`### <number-in-words>`

on its own, where `<number-in-words>` is the final answer in English.

Worked example

Input prompt:

Natalia sold clips to 48 of her friends in April, and then she sold half as many clips in May. How many clips did Natalia sell altogether in April and May?

Note: the user prompt itself contains the digit `48`. The constraint applies to your model's output, not to the user's input.

A passing output (all three components = 1):

Natalia sold forty-eight clips in April. In May she sold half as many, which is twenty-four. Adding the two months together, forty-eight plus twenty-four equals seventy-two clips altogether.

seventy-two

Failing outputs and why:

Output (last line shown)	What's wrong
... ### 72	digits in the answer line
Half of 48 is 24, total 72.	digits in the prose, no ### line
... The answer is seventy-two.	no ### line
forty-8	mixed form — has digit 8
### seventy-two – final answer	extra text after the ### line
... ### seventy	wrong number

What we give you

- Model: Qwen/Qwen2.5-0.5B-Instruct. 500M parameters, fits on a Colab T4. Already instruction-tuned and gets ~30% on GSM8K out of the box. LoRA only (you pick rank and target modules; defend the choice).
- Dataset: openai/gsm8k, config main. ~7,473 train / 1,319 test. Use train for training, a 200-example slice of test for evaluation.
- Algorithm: GRPO via trl.GRPOTrainer. You must set use_vllm=False — vLLM does not fit on a free Colab T4. If you prefer a different RLVR algorithm (RLOO, REINFORCE++, ReST, etc.), you may use it; defend the choice.
- Reference policy: if gpu OOMs don't use reference policy, i.e. beta=0
- Compute: Google Colab Free T4 (16 GB). ≤ 4 hours of wall-clock training. Colab will eventually disconnect you, so save checkpoints.

What to deliver

Three artifacts (zip, repo, or Drive link).

1. train.ipynb

A single notebook that runs top-to-bottom on a fresh Colab Free T4. Includes:

- Install + imports + dataset/model load.
- Your reward function, with comments.
- Training loop with logged metrics (W&B, TensorBoard, or stdout — your choice).
- LoRA weights saved at the end.
- An inline cell that evaluates the trained model on your eval slice.

2. eval_table.md

Run 200 problems from the GSM8K test split through both the base model and your trained model, using the same prompt and decoding settings for both, and report:

Metric	Base	Trained
numeric correctness (matches gold)	?	?
no-digits compliance	?	?
format compliance	?	?
all three together	?	?

Below the table, paste the prompt template and decoding configuration you used.

3. `WRITEUP.md` — the document we read most carefully (≤ 1500 words)

In this exact order, with headings:

1. Reward design. Show your reward function. Explain why this shape. What does each component do? What would go wrong with a different shape (a plain weighted sum, or a multiplicative product)?
2. Length, sampling, and group size. Your `temperature`, `top_p`, `max_completion_length`, and `K` (rollouts per group). Defend each.
3. One failure mode you actually hit and fixed. Symptom in logs, diagnosis, fix.
4. What you would change with a 24-hour H100 instead of a 4-hour T4. Be specific (more rollouts? larger LoRA? different `K`? full finetune? vLLM-backed sampling?).

Rules

- LoRA only. No full finetune.
- No external LLM calls for reward, eval. If you choose to use LLM for anything else, declare it in the writeup.
- The verifier must be deterministic and algorithmic. Regex plus a `word↔int` library is fine. Test it; we will throw weird strings at it during follow-up.

What we are not asking for

- A new RL algorithm.
- SOTA accuracy.
- Paper-quality evaluation; 200 examples is enough.
- A pretty UI.

How we grade

We read `WRITEUP.md` first, then `eval_table.md`, then we open the notebook and re-run a couple of cells. The notebook must execute. The numbers in the table must roughly reproduce. We are looking for:

- A reward shape you can defend, not just one that exists.

- Evidence you noticed and named at least one nontrivial failure mode of this task.
- An eval that compares against the base model, not against zero.
- A writeup that reasons rather than narrates.

A model going from 0% → 30% all-three with a clear writeup beats a model at 50% with hand-waving.

Last but not least, we strictly want to see your reasoning behind how you approach a problem rather than pure numbers.

P.S : LLM is used to improve the language of the document. Please let us know if you encounter any issues. May the Wind be with you
